

Classification Models

Applied Machine Learning — Session 2, Chapter 2

Classification: What We're Doing

Predicting a discrete category.

- **Binary:** spam / not spam · malignant / benign · churn / stay
- **Multi-class:** iris species · digits 0–9 · which of 3 grape cultivars (today's exercise)

Two kinds of output:

```
model.predict(X)          # hard label:  1
model.predict_proba(X)    # probabilities: [0.08, 0.92] → row sums to 1
```

The probability is often the more useful thing (Ch06: we can move the 0.5 threshold).

Baseline First: The Majority Class

```
from sklearn.dummy import DummyClassifier
DummyClassifier(strategy='most_frequent').fit(X_train, y_train).score(X_test, y_test)
```

Breast cancer data: 63 % benign → **the dummy scores 63 % accuracy by always saying "benign".**

Every real model must beat that number — and (spoiler for Ch06) accuracy alone will not tell you *how* good it is.

Logistic Regression — a Classifier, Despite the Name

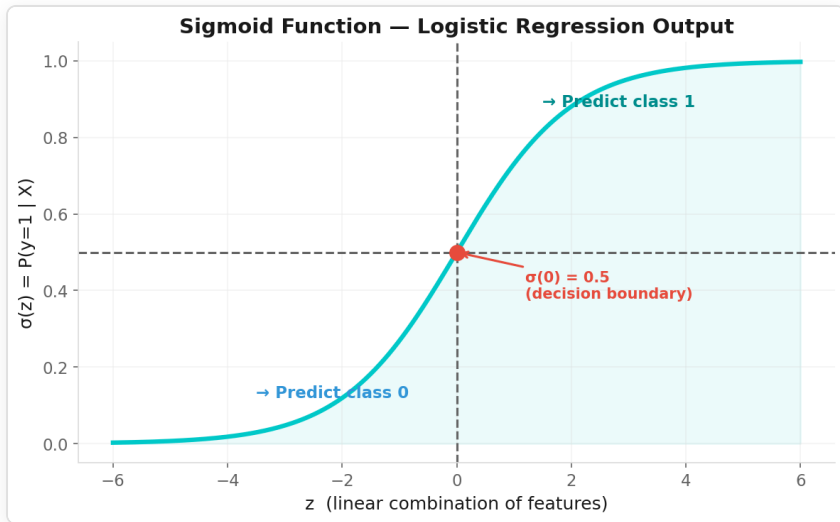
Models the **probability** of class 1 with a linear score squashed through the sigmoid:

$$P(y = 1 \mid x) = \sigma(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n), \quad \sigma(z) = 1 / (1 + e^{-z})$$

```
from sklearn.linear_model import LogisticRegression
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
model.predict_proba(X_test)[: , 1]      # P(class 1)
```

- **Linear decision boundary** — fast, interpretable (β = effect on log-odds), great baseline
- Trained by gradient descent on the *cross-entropy* loss (the classification cousin of MSE)

The Sigmoid



- $\sigma(0) = 0.5 \rightarrow$ the **decision boundary** is where the linear score is 0
- $\sigma(+\text{large}) \rightarrow 1, \sigma(-\text{large}) \rightarrow 0$
- `predict()` = " $P > 0.5$ " — the 0.5 is a *choice* (Ch06)

KNN — 2-Minute Recap from Ch03

"Look at the k nearest training points, take a majority vote."

```
from sklearn.neighbors import KNeighborsClassifier
make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
```

- Distances → **needs scaling** • no real training phase, slow to predict on big data
- **k is the complexity dial**: k = 1 memorizes (overfit), huge k → everything becomes the majority class (underfit)
- Choose k with cross-validation — exactly like degree / α in Ch04

Decision Tree Classifier

A sequence of yes/no questions, chosen to make the resulting groups as *pure* as possible (Gini impurity).

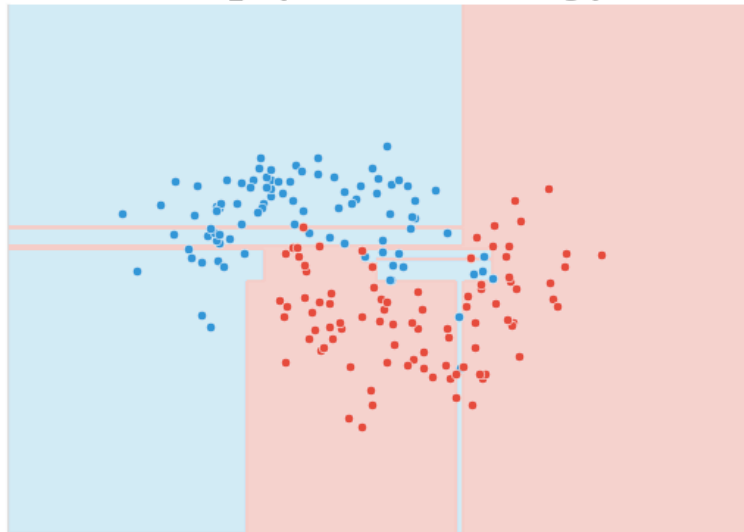
```
petal_length < 2.5 ?  
  yes → setosa ✓  
  no  → petal_width < 1.8 ?   yes → versicolor ✓ / no → virginica ✓
```

```
from sklearn.tree import DecisionTreeClassifier, plot_tree  
dt = DecisionTreeClassifier(max_depth=3, random_state=42).fit(X_train, y_train)  
plot_tree(dt, feature_names=feature_names, class_names=class_names, filled=True)
```

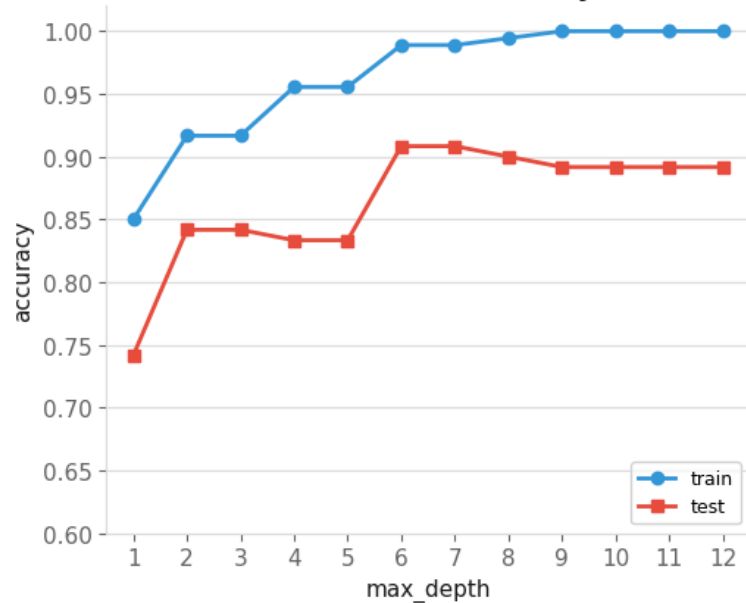
Fully interpretable — every prediction is a readable rule. No scaling needed.

The Complexity Dial Again: Tree Depth

DecisionTree(max_depth=12) — training points shown



Train vs. test accuracy



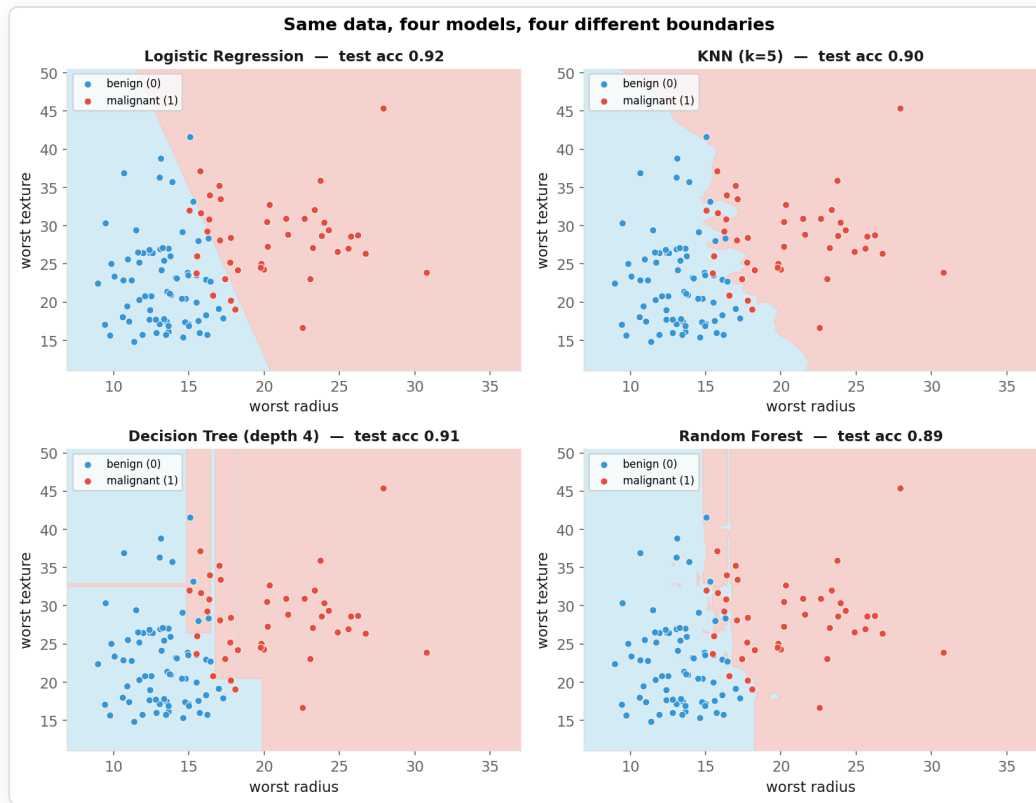
Random Forest Classifier

Many deep trees, each on random rows + random feature subsets → majority vote.

```
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42).fit(X_train, y_train)
pd.Series(rf.feature_importances_, index=feature_names).nlargest(10).plot.barh()
```

- Averaging many over-fitted trees **cancels their individual noise** → low variance, rarely overfits badly
- Handles mixed feature types, no scaling, gives **feature importances**
- Almost always beats a single tree; costs interpretability

Decision Boundaries: What Each Model Learned



Before the Exercise: Reading a Confusion Matrix

	Predicted: healthy (0)	Predicted: cancer (1)
Actual: healthy (0)	TN healthy, predicted healthy ✓ correct	FP healthy, predicted cancer false alarm (Type I)
Actual: cancer (1)	FN cancer, predicted healthy MISSED CASE (Type II) △	TP cancer, predicted cancer ✓ correct

rows = truth, columns = prediction (sklearn convention) — positive class = 1 = the thing you look for

```
ConfusionMatrixDisplay.from_predictions(y_test, y_pred, display_labels=class_names)
print(classification_report(y_test, y_pred))    # precision / recall / F1 per class
```

Quick Check

1. A tumour is malignant, the model says "benign". Which cell of the confusion matrix is that — and is it the expensive error?
2. Your KNN accuracy jumps from 0.72 to 0.95 after adding `StandardScaler`. Why?
3. Which model would you pick if the customer must be able to *explain every decision* to a regulator?

Quick Check

1. A tumour is malignant, the model says "benign". Which cell of the confusion matrix is that — and is it the expensive error?
 2. Your KNN accuracy jumps from 0.72 to 0.95 after adding `StandardScaler`. Why?
 3. Which model would you pick if the customer must be able to *explain every decision* to a regulator?
-
1. **False negative** (true = 1, predicted = 0) — yes: a missed cancer is far worse than a false alarm
 2. One large-range feature dominated the distance; scaling makes every feature count equally
 3. **Decision tree** (or logistic regression) — not the forest or KNN

Now: Exercises!

→ Open `03-exercises/ch05_classification_exercises.ipynb`

Dataset: `load_wine` — 178 wines, 13 chemical measurements, **3 grape cultivars** (multi-class!)

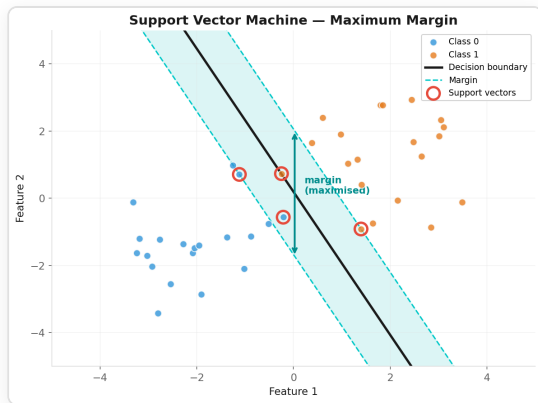
Task	Content
Task 1	baseline + logistic regression pipeline (3 min)
Task 2	KNN pipeline (2 min)
Task 3	random forest + <code>classification_report</code> (4 min)
Task 4	confusion matrix of a <i>weak</i> tree — which classes get mixed up? (3 min)
Bonus	5-fold CV comparison · SVM

Key Takeaways

- **Baseline first** — the majority class sets the floor
- Logistic Regression: linear boundary, outputs probabilities, strong baseline
- KNN: local, needs scaling, k = complexity dial
- Decision Tree: readable rules; depth = complexity dial • Random Forest: many trees, robust, importances
- Same data → very different boundaries; more complex $\neq$ better
- Confusion matrix: rows = truth, positive class = what you look for

Appendix: Support Vector Machines (Bonus)

Find the separating boundary with the widest margin to the nearest points (the *support vectors*).



```
from sklearn.svm import SVC
make_pipeline(StandardScaler(), SVC(kernel='rbf', C=1.0)) # rbf = smooth curved boundaries
```

C small → wide margin (more regularization) • **rbf** → non-linear • needs scaling • slow beyond ~50k rows

Next: Chapter 6

Metrics & Evaluation

"Which model is actually better? It depends on how you measure it."